How to Run Durin's Code

This guide provides step-by-step instructions for building the project from source, running the test suite, and executing the compiled binaries (with all available CLI options) on both Linux and Windows.

---

1. Prerequisites

Ensure your system meets the following requirements before building:

Tool	Version
CMake	3.23
C++ compiler	C++17 (Clang 14+ / GCC 11+)
GoogleTest	system-installed (`libgtest-dev`)

GoogleTest Install (Ubuntu / macOS)

Ubuntu:

```
sudo apt install libgtest-dev
```

macOS (Homebrew):

```
brew install googletest
```

(Note for Windows users: If you are using MSYS2 MinGW, use `pacman -S mingw-w64-x86_64-gtest`. If you are using Visual Studio, it can be installed via `vcpkg install gtest:x64-windows`)

---

2. Build

To compile the project from the root `DurinsCode-main` directory:

```
cd "Compiler construction project/DurinsCode-main"
cmake -S . -B build
cmake --build build
```

(On Windows using MSVC, if you get linking errors, append `--config Release` to the build command: `cmake --build build --config Release`)

Binaries produced in `build/`:

Binary	Purpose
`durinsc`	Main compiler + runtime CLI

Binary	Purpose
`lexer_tests`	Lexer test suite
`parser_tests`	Parser test suite
`semantic_tests`	Semantic analyser test suite
`tac_tests`	TAC/IR test suite
`codegen_tests`	Code generation test suite
`vm_tests`	Virtual machine test suite

---

3. Running Tests

Once built, you can run the entire test suite via `ctest`:

```
cd build
ctest --output-on-failure
```

Or run individual test suites directly: **Linux / macOS / WSL:**

```
./lexer_tests
./parser_tests
./semantic_tests
./tac_tests
./codegen_tests
./vm_tests
```

Windows (PowerShell/CMD):

```
.\lexer_tests.exe
.\parser_tests.exe
# ... and so on ...
```

---

4. Running Examples & CLI Options

The compiler comes with several examples located in the `examples/` directory. All commands below assume you are inside the `build/` directory.

*(**Windows Users:** Replace `./durinsc` with `.\durinsc.exe` and use backslashes `..\examples\...`)*

A. Compile and Run Immediately

This is the standard mode. It compiles the source file and immediately launches the text adventure Virtual Machine.

```
./durinsc ../examples/04_middle_earth.dc
```

Try playing it! Type the following commands when the game starts: `take ring` → `go east` → `go east` → `destroy ring`

B. Compile to Bytecode File (`-o`)

This mode runs the compiler pipeline (Lexer → Parser → Semantic → TAC → Optimizer → Codegen) and saves the fully compiled JSON bytecode to a file, without launching the game.

```
./durinsc ../examples/02_inventory.dc -o inventory.json
```

(You will see a message: "Bytecode written to inventory.json")

C. Run from Pre-Compiled Bytecode (`--run`)

Bypass the compiler entirely and launch the Virtual Machine directly using an already compiled `.json` bytecode file.

```
./durinsc --run inventory.json
```

D. Debug Mode (`--debug`)

Prints the compiler's heavily-detailed internal state before executing. It dumps the **Symbol Table** (showing all parsed rooms, items, and actions) and the **Optimized TAC** (Three-Address Code IR instructions).

```
./durinsc ../examples/01_hello_world.dc --debug
```

(You can combine this with `-o` to debug while compiling to bytecode!)

E. Interactive REPL Mode (`--interactive`)

Starts a Read-Eval-Print Loop where you can type Durin's Code directly into the terminal, hit Enter twice, and see it compile and run instantly.

```
./durinsc --interactive
```

---

5. Overview of Example Files

If you want to test various features of the compiler, run any of these files:

- `../examples/01_hello_world.dc` – The simplest valid program. Good for basic testing.
- `../examples/02_inventory.dc` – Tests `+=` and `-=` inventory logic, and `has_item` conditions.
- `../examples/03_multiroom.dc` – Tests a 4-room graph with movement exits.
- `../examples/04_middle_earth.dc` – **The full comprehensive demo.** Best for demonstrating to reviewers.

- `../examples/05_error_demo.dc` – Intentionally broken program. Run this to test the Semantic Analyzer: `bash ./durinsc ../examples/05_error_demo.dc` *(Expected output: "Compilation failed: 5 error(s).")*